事件流与图书馆管理系统 UML 建模

信息系统分析与设计（ISAD）课程作业

学　　号:	2441502005
姓　　名:	徐吉涛
提交日期:	2026 年 10 月 30 日
建模语言:	UML 2.5

> 本文所有图形均以 PlantUML 描述并渲染，源码保存在同名目录的 `puml/` 下，可逐图复现。图号与图片文件一一对应。

一、微信小程序「修改用户名 / 密码」操作的事件流

1.1 场景界定

以校园常用的微信小程序（图书馆、教务、校园卡一类）的账户安全模块为例。用户从「我的 — 账号与安全」入口发起登录密码（或绑定用户名）的变更。这类操作具有三个特征：**一次性**（不能反悔、不产生中间草稿）、**高风险**（直接决定账户归属）、**需二次核验**（不能仅凭已登录状态就允许修改）。

事件流（Event Flow）关注"发生了什么"，按时间顺序把用户动作、系统命令、领域事件与对外可见结果串成一条链，不纠缠内部实现细节。它介于用例描述与序列图之间：比用例更细，比序列图更聚焦业务语义。

1.2 事件流主线

#	事件	类型	发出者	系统响应	对外可见结果
1	进入「账号与安全」页	界面事件	用户	拉取账户安全信息	显示绑定手机、最近登录设备
2	点击「修改登录密码」	用户命令	用户	弹出二次核验方式选择	出现「微信授权 / 短信验证码」
3	完成微信授权或短信校验	系统命令	小程序前端	调用微信开放平台换取 openid 与一次性凭证	显示核验中

4	凭证核验通过	系统事件	用户服务	建立 5 分钟有效的变更会话	进入新密码输入页
5	提交新密码	用户命令	用户	强度校验，并与旧密码比对	弱密码即时提示
6	写入新密码哈希	持久化事件	用户服务	bcrypt 哈希覆盖，账户版本号 +1	无
7	PasswordChanged	领域事件	用户服务	吊销该账号全部登录令牌	其他设备被强制下线
8	变更通知	集成事件	通知服务	下发站内信与微信订阅消息	用户收到「密码已修改」提醒
9	强制重登	界面事件	小程序前端	清除本地 token，跳转登录页	回到登录页

1.3 事件流图

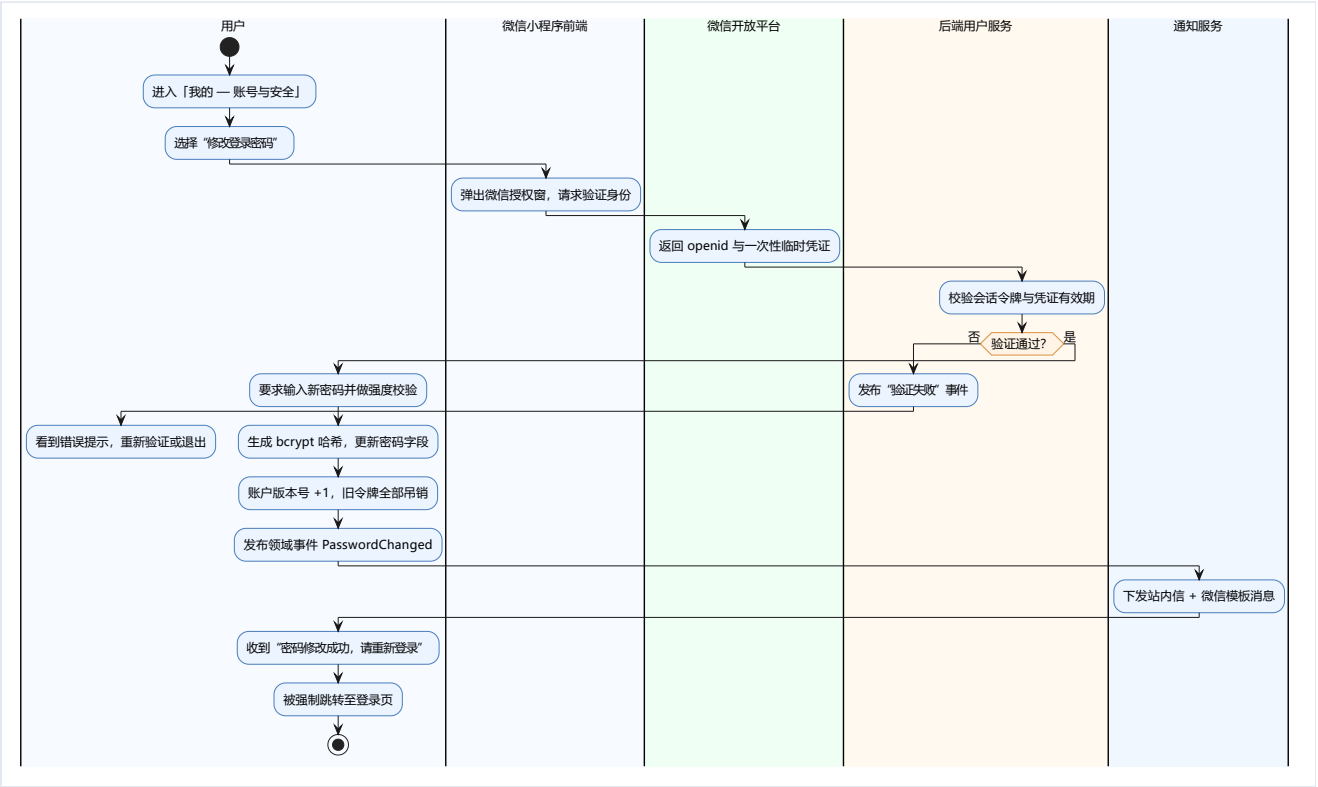


图 1　微信小程序「修改登录密码」事件流（含泳道的活动图）

事件流要点

- **单一入口、单一出口**：无论成功还是失败，流程都不回流到同一节点，避免用户反复尝试形成死循环。
- **核验前置**：密码变更必须先通过二次核验，"已登录"本身不构成修改授权，这是整个流程的安全闸门。
- **事件与副作用分离**：密码写入是主事务，而"通知"和"踢下线"由 PasswordChanged 事件驱动，作为副作用异步执行，便于审计与扩展。
- **可追溯**：每次变更形成一条审计记录（时间、IP、设备、操作类型），支撑异常登录回溯。

二、图书馆管理系统建模

系统边界：面向高校图书馆的借阅管理子系统，覆盖读者端（微信小程序 / Web OPAC）、馆员端（编目与流通）与后台管理。核心业务对象为书目、馆藏副本、借阅记录、预约记录与罚金。

2.1 序列图 Sequence Diagram

用途 描述「借阅图书」这一核心用例中，参与者与系统对象之间按时间先后排列的消息交互，重点在于**顺序**与**分支**。

下表按时间顺序列出该用例的消息交互；「借阅图书」共涉及三个数据源，任一步失败即回滚并返回原因码。

序号	发送方 → 接收方	消息	说明
1	读者 → 小程序前端	扫描图书条码 / 输入 ISBN	发起借阅申请
2	前端 → 借阅服务	borrowBook(readerId, copyBarcode)	携带读者身份与副本条码
3	借阅服务 → 图书目录库	查询副本状态与馆藏位置	确认是否可借
4	图书目录库 → 借阅服务	副本 BC-0007 在架，可借	返回状态
5	借阅服务 → 借阅记录库	查询在借册数、逾期与罚金	校验借阅资格
6	借阅记录库 → 借阅服务	在借 2 册（上限 5 册），无逾期	校验通过
7	借阅服务 → 图书目录库	置副本状态为「已借出」	锁定副本
8	借阅服务 → 借阅记录库	创建借阅记录（应还日期 +30 天）	返回 loanId
9	借阅服务 → 前端	借阅成功，应还 2026-11-29	回传凭证
10	借阅服务 → 通知服务	sendLoanNotice(loanId)	异步触发
11	通知服务 → 读者	微信模板消息「借阅成功」	外部可见结果
alt	借阅服务 → 前端	失败原因码 E_BORROW_LIMIT	额度 / 逾期 / 副本不可借时走此分支
opt	借阅服务 → 借阅记录库	关闭预约记录，标记「已满足」	读者此前预约过该书时执行

组合片段：alt 表示互斥分支（校验通过 / 失败），opt 表示可选执行（存在预约才关闭），与序列图的语义一一对应。

- 借阅是一次**跨三个数据源**的事务：目录库（副本状态）、借阅记录库（额度与逾期）、通知服务。
- 用 `alt` 片段表达"校验通过 / 失败"两条互斥分支，用 `opt` 片段表达"若存在预约则一并关闭"的可选逻辑。
- 副本状态变更与借阅记录创建必须同事务，或采用最终一致加补偿，否则会出现"书被借走却没有记录"的账实不符。

2.2 用例图 Use Case Diagram

用途 划定系统功能边界，识别参与者与可用功能，并用 include / extend 表达功能之间的依赖。

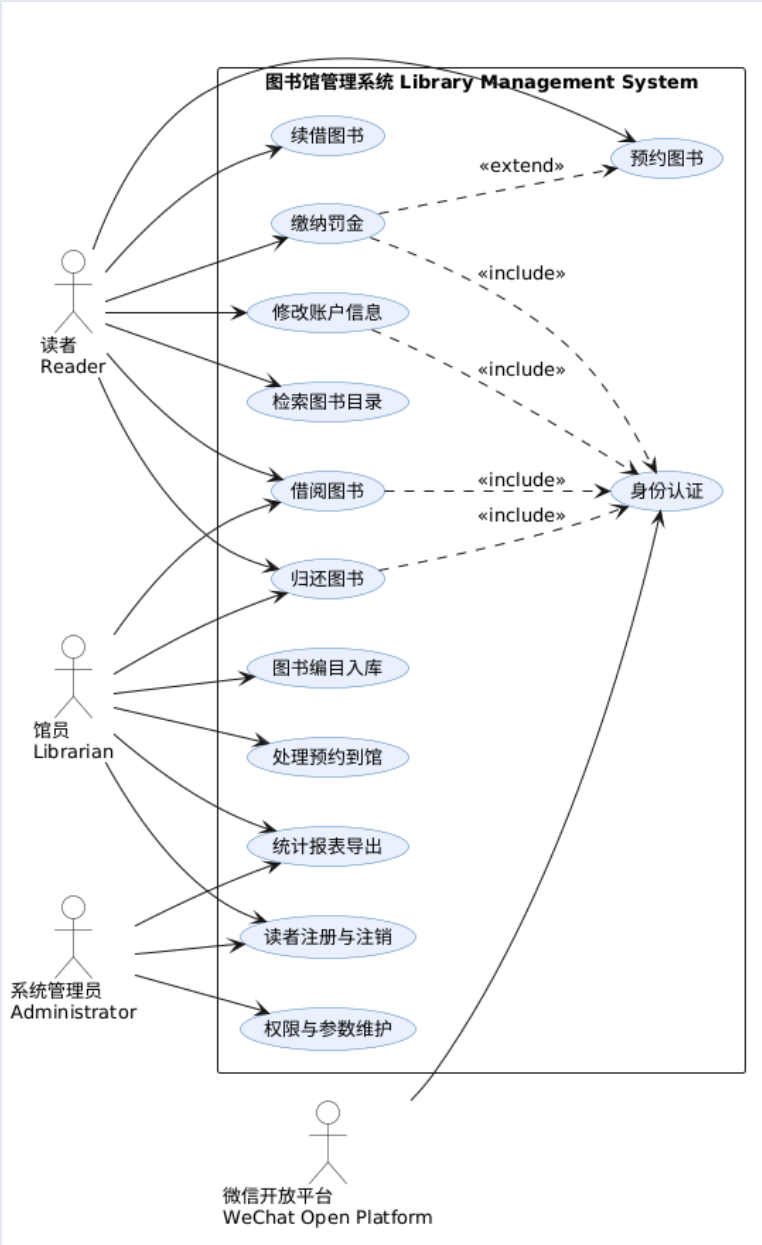


图 2　图书馆管理系统用例图

- 三类人类参与者：读者（自助服务）、馆员（业务操作）、系统管理员（治理）；一个外部系统参与者：微信开放平台。
- **身份认证**被四个用例 include，说明它是横切关注点，应实现为公共能力，而非在每个用例里重复实现。
- 「缴纳罚金」对「预约图书」是 extend：仅在"先缴清罚金才能预约"的规则被触发时调用，属于条件性扩展，不是必然包含。

2.3 类图 Class Diagram

用途 刻画领域静态结构——实体、属性、操作与关联的多重性，是数据库表结构设计的直接依据。

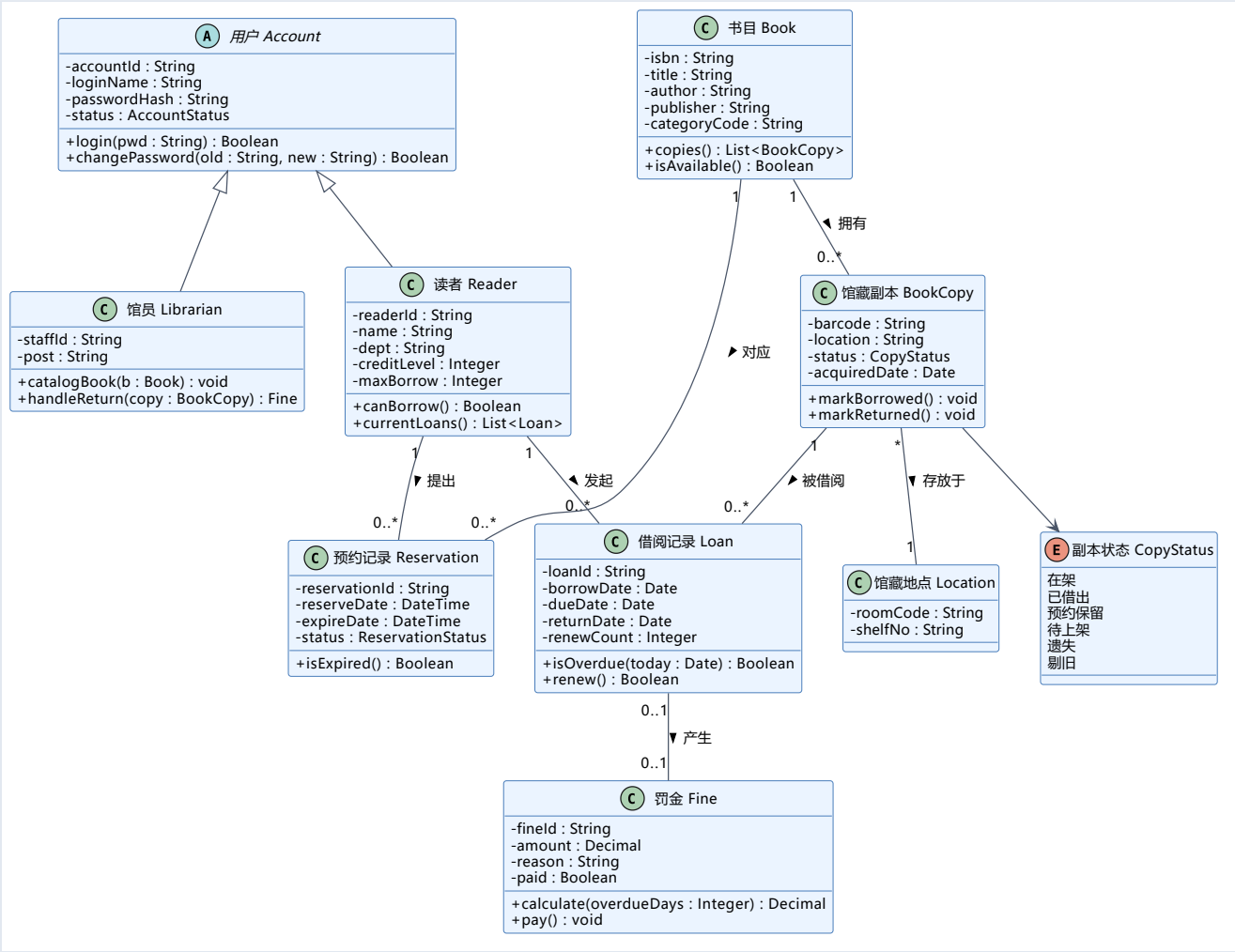


图 3　图书馆管理系统类图

- **书目 Book 与馆藏副本 BookCopy 分离**（1 : 0..*）是图书馆领域最关键的建模决策：一种书有多个可独立借还的物理副本，题名信息不必重复存储。
- 抽象类 Account 抽取读者与馆员的公共属性与操作（登录、改密），正对应图 1 的账户事件流。
- Loan 保证借阅历史的可追溯；Fine 与 Loan 为 0..1 对 0..1，仅在逾期或损毁时产生。
- CopyStatus 用枚举表达状态取值，与状态图（图 8）一一对应，避免状态字符串散落在代码中。

2.4 对象图 Object Diagram

用途 类图在**某一时刻**的快照，用具体实例验证类图的多重性与关联是否真的成立。

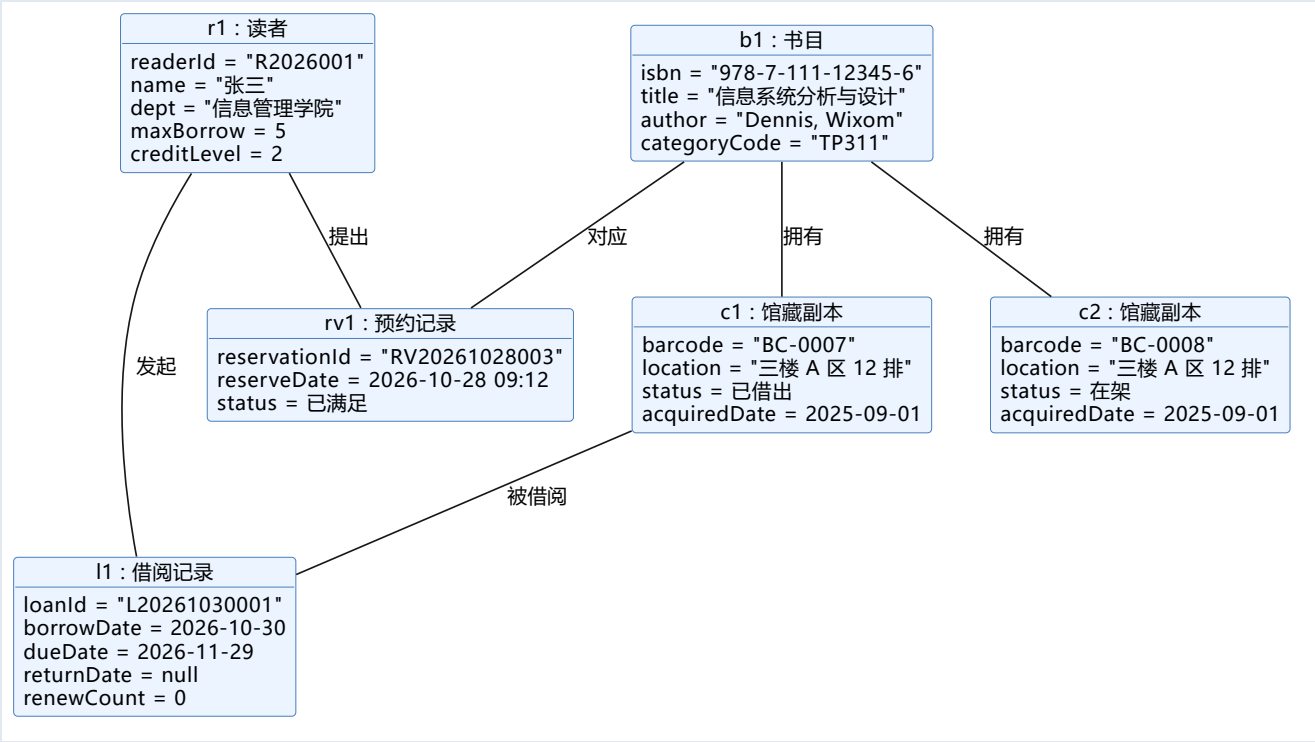


图 4　借阅场景对象图（2026-10-30 时刻）

- 同一书目 b1 下挂两个副本 c1（已借出）与 c2（在架），直观验证"1 书目 : N 副本"的多重性。
- 读者 r1 与其借阅记录 l1、预约记录 rv1 构成实例链，呈现一次借阅背后的完整对象网络。
- 对象中 status = 已借出 正是状态图（图 8）中的一个状态取值，两张图互相印证。

2.5 活动图 Activity Diagram

用途 描述「借阅图书」的业务流程与控制流，含泳道划分与多重判断节点，关注**规则**如何在流程中被逐层过滤。

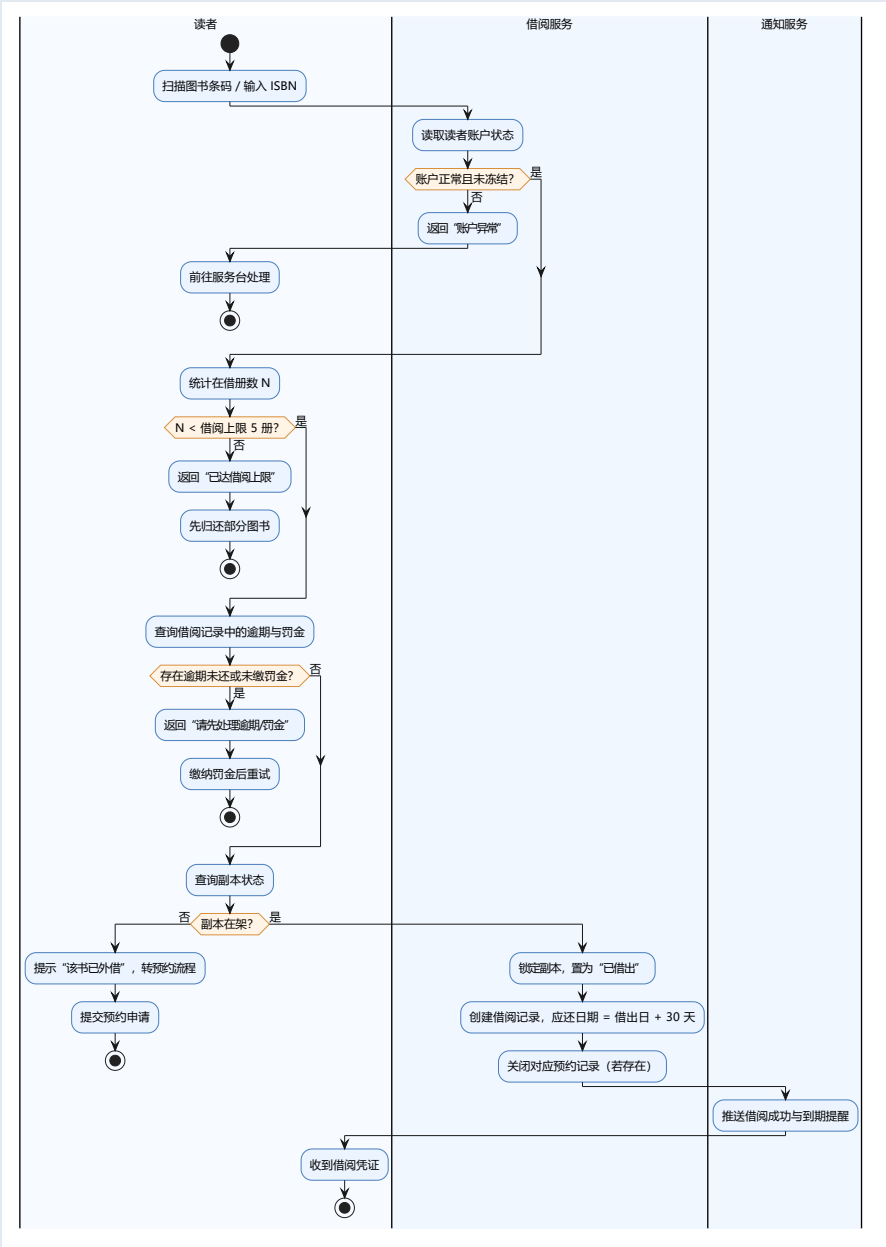


图 5　借阅业务活动图（三泳道）

- 三个泳道划分职责：读者、借阅服务、通知服务，明确"谁做什么"。
- 四道连续判断构成借阅规则集：账户状态、借阅额度、逾期与罚金、副本是否在库；任一不通过即快速失败，并给出可操作的提示。
- 借阅失败并非简单终止，而是引导至**预约流程**，把异常路径转化为可留存的业务机会。

2.6 遗留语法图 Legacy Syntax

用途 PlantUML 在 2018 年推出新版活动图引擎之前，使用（*）--> 形式的旧语法（Legacy Activity Diagram）。旧语法至今仍被兼容支持，特点是**以箭头连线为主、节点作为箭头指向的对象**，适合表达线性、分支简单的流程。

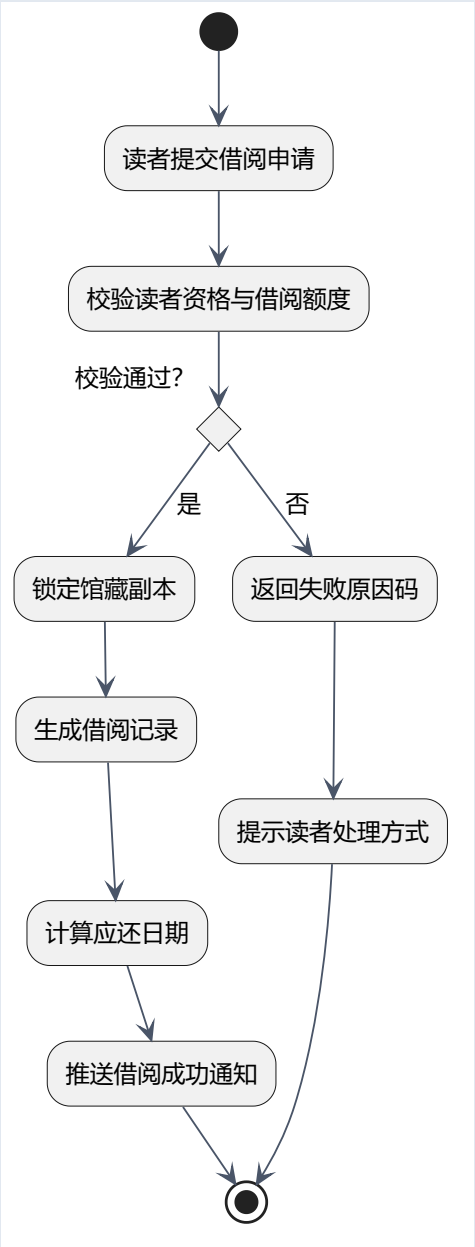


图 6　用旧语法（(*) -->）绘制的借阅流程

- 旧语法用 (*) 表示初始与结束节点，节点名后直接接 -->[标签] "下一节点"；新语法用 start / stop 与 :活动描述; 三段式。
- 旧语法**不支持泳道**，也难以表达嵌套的并行分叉，因此图 5 这类复杂业务必须使用新语法。
- 保留旧语法的现实意义：大量历史文档、教学示例与社区代码仍在使用，能读懂才谈得上维护。

语义	Legacy 语法	现行语法（Activity Beta）
起始节点	(*)	start
终止节点	(*)	stop / end
活动节点	"节点名" 作为箭头目标	:活动描述;
分支	if "条件" then … else … endif	if (条件) then (是) … endif

泳道	不支持	|泳道名|

2.7 组件图 Component Diagram

用途 从实现视角划分系统模块、对外接口与依赖关系，指导服务拆分与团队分工。

组件	职责	主要依赖
微信小程序 / Web OPAC	读者端界面：检索、借阅、预约、账户	API 网关
API 网关 / 统一认证	路由、限流、鉴权，收敛所有外部访问	七个业务服务
图书检索服务	书目查询、分类导航、馆藏定位	MySQL
借阅服务	借出、归还、续借、逾期判定	检索服务、账户服务、MySQL、消息队列
预约服务	预约排队、到馆保留、期满释放	MySQL、消息队列
读者账户服务	登录、改密、额度与信用等级	MySQL、微信开放平台
罚金结算服务	逾期罚金计算、缴纳与减免	MySQL
通知服务	消费领域事件，下发订阅消息与站内信	消息队列、微信开放平台
报表统计服务	借阅量、热门书目、读者行为统计	MySQL（只读副本）

组件之间通过**消息队列**解耦：借阅服务发布 LoanCreated、预约服务发布 ReservationReady，通知服务异步消费——这正是事件流在架构层的落地方式。

- 按**业务能力**而非技术层次拆分：检索、借阅、预约、账户、罚金、通知、报表七个服务组件，各自可独立部署。
- 所有外部访问经「API 网关 / 统一认证」收敛，避免认证逻辑在七个服务中重复实现。
- 借阅与预约通过消息队列发布领域事件（LoanCreated、ReservationReady），通知服务异步消费——这正是图 1 事件流在架构层的落地方式。

2.8 部署图 Deployment Diagram

用途 描述物理部署结构：节点（Node）、运行其上的制品（Artifact），以及节点之间的通信协议与端口。

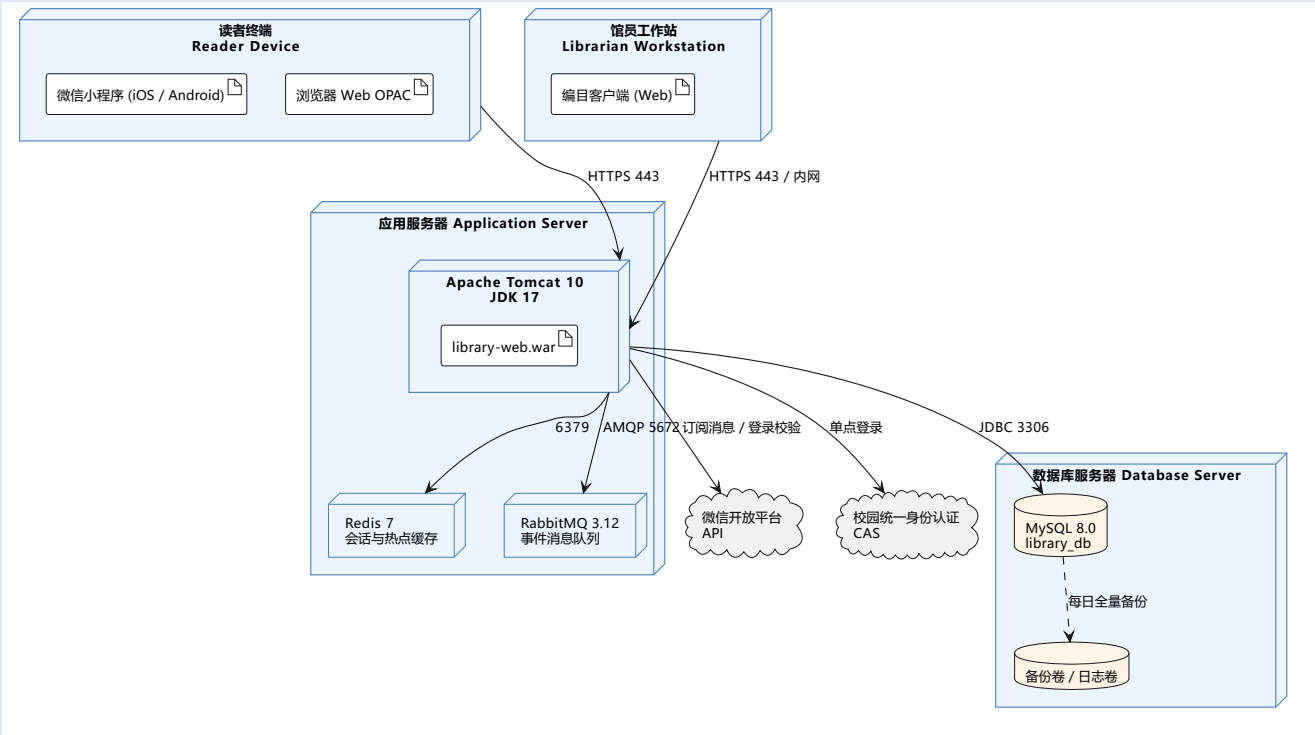


图 7　图书馆管理系统部署图

- 三层结构：终端（读者手机 / 浏览器、馆员机）、应用服务器（Tomcat + Redis + MQ）、数据库服务器。
- 端口与协议在图上标注（HTTPS 443、JDBC 3306、Redis 6379、AMQP 5672），可直接作为运维防火墙策略的依据。
- 两个外部依赖：微信开放平台（消息与登录）与校园统一身份认证 CAS，说明本系统并非孤立运行，而是校园信息化体系的一环。

2.9 状态图 State Diagram

用途 刻画**单个馆藏副本**在生命周期内的状态迁移，是类图中 CopyStatus 枚举的行为补充。

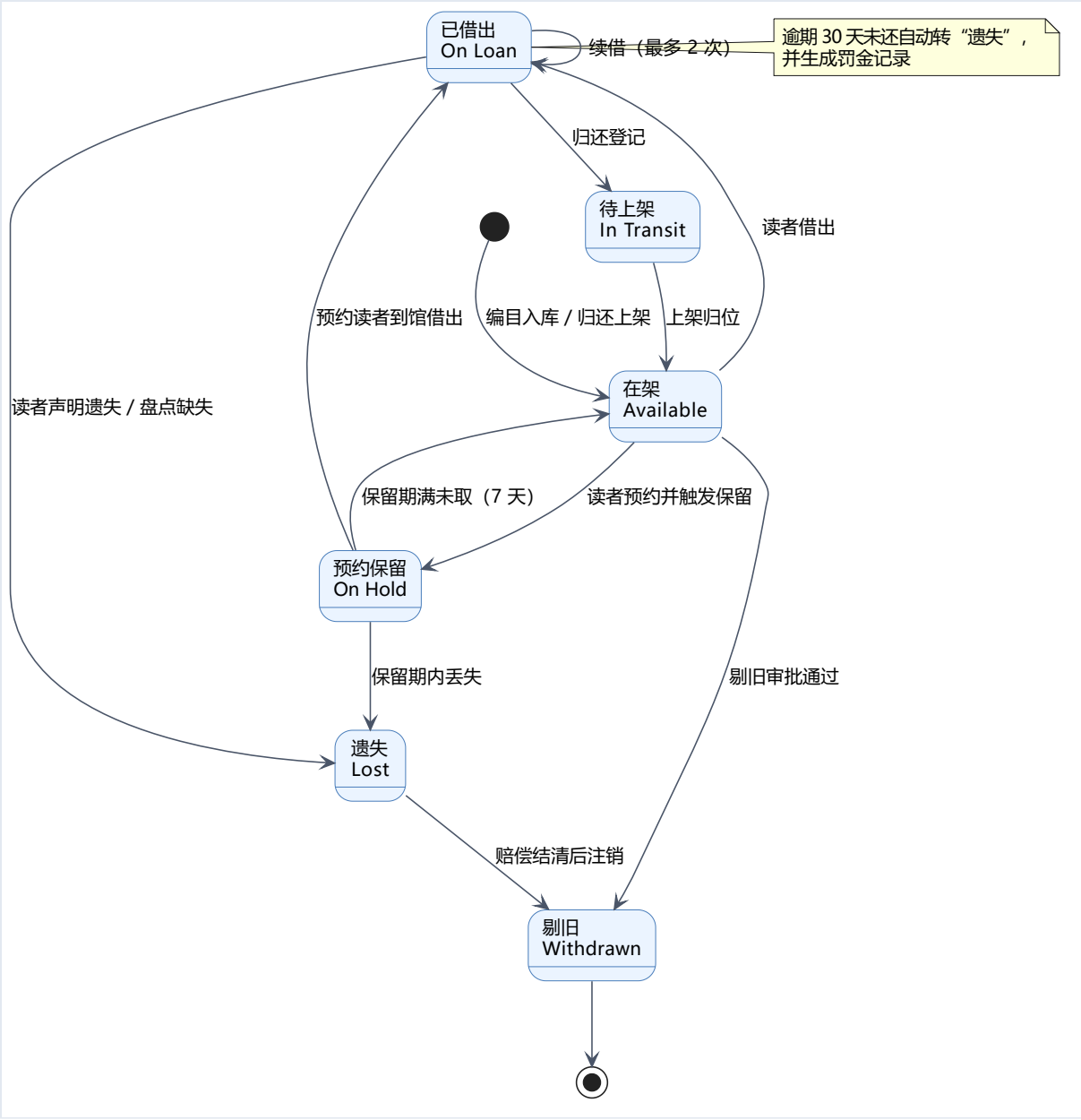


图 8　馆藏副本 BookCopy 状态图

- 六个状态：在架、预约保留、已借出、待上架、遗失、剔旧；初始进入点为"在架"（编目入库后立即可借）。
- 「已借出 → 已借出」的自环表示**续借**：状态不变，但应还日期与续借次数发生变化。
- 异常出口被显式建模：逾期 30 天自动转"遗失"并生成罚金，避免"书已丢失、系统仍显示在借"的账实不符。

2.10 时序图 Timing Diagram

用途 以时间轴为横轴，展示一个副本实例与一次借阅随**时间推移**的状态变化，是状态图在时间维度上的展开。

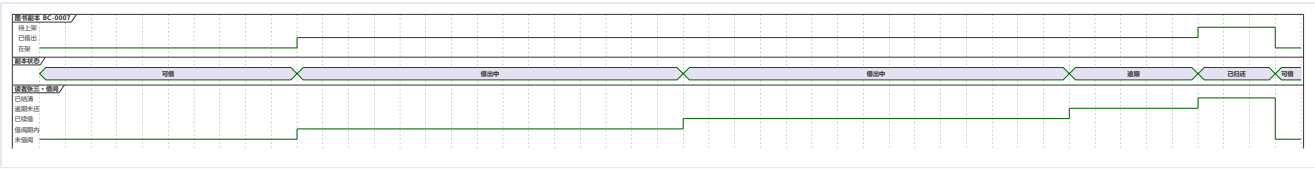


图 9　副本与借阅关系的时序图（Timing）

- 横轴为天数（第 0 天至第 48 天），三条泳道分别为副本本体、副本状态、读者借阅关系。
- robust 用波形表达取值变化，concise 用色块表达区间状态，两种视图对照即可同时看到"变化时刻"与"持续区间"。
- 第 25 天续借、第 40 天逾期、第 45 天归还三个时间点被显式标注，可作为超期提醒与罚金计算的规则依据。

三、模型之间的关系

十张图并非彼此独立，而是同一系统的不同切面，可沿三条线索串联：

- **需求 → 结构**：用例图（图 2）确定"借阅图书"是系统应提供的功能，序列图（图 2）说明它如何交互，类图（图 3）与对象图（图 4）给出它涉及的数据结构。
- **结构 → 行为**：类图中的 CopyStatus 枚举，在状态图（图 8）中被展开为完整的迁移规则；状态图中的"借出中"区间，又在时序图（图 9）中沿时间轴铺开。
- **行为 → 实现**：活动图（图 5）的每道判断对应服务端一条业务规则，组件图（图 8）把这些规则分配到不同服务组件，部署图（图 7）再决定它们运行在哪台机器上。

图 1 的账户事件流，则是类图中 Account.changePassword() 操作的时序细化，也是组件图中「读者账户服务 → 微信开放平台」这条依赖的业务来源。同一套业务，四种视角，互相校验。

附录　图形清单

图号	名称	类型	说明
图 1	修改密码事件流	Activity（泳道）	小程序账户安全操作的事件链
图 2	借阅图书	Sequence	跨三数据源的交互时序
图 2	系统功能边界	Use Case	三类参与者与 13 个用例
图 3	领域静态结构	Class	书目与副本分离的经典建模
图 4	借阅场景快照	Object	类图的实例验证
图 5	借阅业务流程	Activity	四道借阅规则判断
图 6	借阅流程（旧写法）	Legacy Activity	(*) --> 旧语法对照
图 8	服务组件划分	Component	七服务 + 网关 + 消息队列

图 7	物理部署结构	Deployment	三层节点与端口
图 8	馆藏副本生命周期	State	六状态的迁移规则
图 9	副本状态时间轴	Timing	续借 / 逾期 / 归还的时间点